

```

import pandas as pd
import numpy as np

from sklearn.datasets import fetch_openml
mnist = fetch_openml("mnist_784", version=1, as_frame=False)
mnist.keys()

raw_data, raw_labels = mnist["data"], mnist["target"]

raw_labels.shape
raw_labels

array(['5', '0', '4', ..., '4', '5', '6'], shape=(70000,),
dtype=object)

raw_dataframe = pd.DataFrame(raw_data)
raw_dataframe.to_csv("raw_data.csv")
raw_labelsframe = pd.DataFrame(raw_labels)
raw_labelsframe.to_csv("raw_labels.csv")

print(mnist["DESCR"])

**Author**: Yann LeCun, Corinna Cortes, Christopher J.C. Burges
**Source**: [MNIST Website](http://yann.lecun.com/exdb/mnist/) - Date
unknown
**Please cite**:

```

The MNIST database of handwritten digits with 784 features, raw data available at: <http://yann.lecun.com/exdb/mnist/>. It can be split in a training set of the first 60,000 examples, and a test set of 10,000 examples

It is a subset of a larger set available from NIST. The digits have been size-normalized and centered in a fixed-size image. It is a good database for people who want to try learning techniques and pattern recognition methods on real-world data while spending minimal efforts on preprocessing and formatting. The original black and white (bilevel) images from NIST were size normalized to fit in a 20x20 pixel box while preserving their aspect ratio. The resulting images contain grey levels as a result of the anti-aliasing technique used by the normalization algorithm. the images were centered in a 28x28 image by computing the center of mass of the pixels, and translating the image so as to position this point at the center of the 28x28 field.

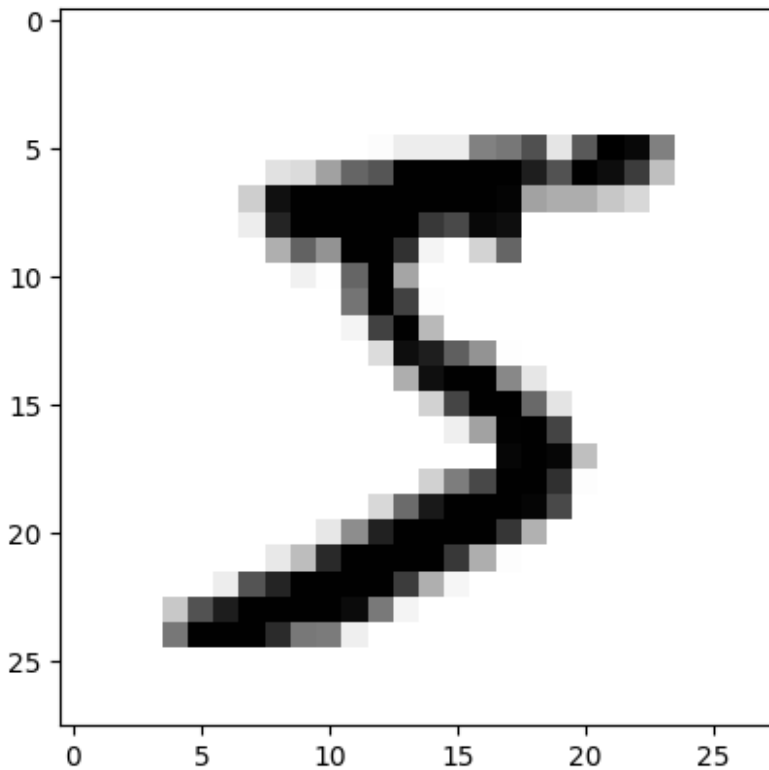
With some classification methods (particularly template-based methods, such as SVM and K-nearest neighbors), the error rate improves when the digits are centered by bounding box rather than center of mass. If you do this kind of pre-processing, you should report it in your publications. The MNIST database was constructed from NIST's NIST originally designated SD-3 as their training set and SD-1 as their test set. However, SD-3 is much cleaner and easier to recognize than

SD-1. The reason for this can be found on the fact that SD-3 was collected among Census Bureau employees, while SD-1 was collected among high-school students. Drawing sensible conclusions from learning experiments requires that the result be independent of the choice of training set and test among the complete set of samples. Therefore it was necessary to build a new database by mixing NIST's datasets.

The MNIST training set is composed of 30,000 patterns from SD-3 and 30,000 patterns from SD-1. Our test set was composed of 5,000 patterns from SD-3 and 5,000 patterns from SD-1. The 60,000 pattern training set contained examples from approximately 250 writers. We made sure that the sets of writers of the training set and test set were disjoint. SD-1 contains 58,527 digit images written by 500 different writers. In contrast to SD-3, where blocks of data from each writer appeared in sequence, the data in SD-1 is scrambled. Writer identities for SD-1 is available and we used this information to unscramble the writers. We then split SD-1 in two: characters written by the first 250 writers went into our new training set. The remaining 250 writers were placed in our test set. Thus we had two sets with nearly 30,000 examples each. The new training set was completed with enough examples from SD-3, starting at pattern # 0, to make a full set of 60,000 training patterns. Similarly, the new test set was completed with SD-3 examples starting at pattern # 35,000 to make a full set with 60,000 test patterns. Only a subset of 10,000 test images (5,000 from SD-1 and 5,000 from SD-3) is available on this site. The full 60,000 sample training set is available.

Downloaded from openml.org.

```
%matplotlib inline
import matplotlib as mpl
import matplotlib.pyplot as plt
some_digit = raw_data[0]
some_digit_image = some_digit.reshape(28, 28)
plt.imshow(some_digit_image, cmap = mpl.cm.binary,
interpolation="nearest")
plt.axis("on")
plt.grid(False)
plt.show()
```



```
raw_labels[11]
'5'
```

We need to convert the string type labels to numerical data type as almost any machine learning model tends to perform better in numerical data compared to string or object datatype

```
raw_labels = raw_labels.astype(np.uint8)
raw_labels[11]
np.uint8(5)
```

Dividing the data into train set and test set. For our dataset the data is already divided into train set and test set. The first 60000 data are train set rest 10000 dataset is test set

```
X_train, X_test, y_train, y_test = raw_data[:60000], raw_data[60000:],
raw_labels[:60000], raw_labels[60000:]
```

Firstly we create a binary classifier which identifies if a digit is 5 or not before jumping to the actual model

```
y_train_5 : bool = []
y_test_5 : bool = []
```

```

for i in range(len(y_train)):
    if y_train[i] == 5:
        y_train_5.append(True)
    else:
        y_train_5.append(False)

for i in range(len(y_test)):
    if y_test[i] == 5:
        y_test_5.append(True)
    else:
        y_test_5.append(False)

y_train_5_book = (y_train == 5)
y_test_5_book = (y_test == 5)
print(type(y_train_5))
print(type(y_test_5_book))
print(type(y_train_5_book))
print(type(y_test_5))

<class 'list'>
<class 'numpy.ndarray'>
<class 'numpy.ndarray'>
<class 'list'>

import joblib
from sklearn.linear_model import SGDClassifier
try:
    sgd_clf = joblib.load("sgd_model_predict_5.pkl")
    print("Model imported from saved models")
except Exception as e:
    sgd_clf = SGDClassifier(random_state=42)
    sgd_clf.fit(X_train, y_train_5_book)
    joblib.dump(sgd_clf, "sgd_model_predict_5.pkl")

```

Model imported from saved models

d:\Code Playground\Python\AI_ML\classificationDigits\.venv\Lib\site-packages\sklearn\base.py:380: InconsistentVersionWarning: Trying to unpickle estimator SGDClassifier from version 1.5.0 when using version 1.6.1. This might lead to breaking code or invalid results. Use at your own risk. For more info please refer to: https://scikit-learn.org/stable/model_persistence.html#security-maintainability-limitations

```
warnings.warn(
```

```

a_number = X_train[:15]
print(type(a_number))
sgd_clf.predict(a_number)

<class 'numpy.ndarray'>

```

```

array([ True, False, False, False, False, False, False, False, False,
       False, False,  True, False, False, False])

from sklearn.model_selection import StratifiedKFold
from sklearn.base import clone

skfolds = StratifiedKFold(n_splits=3, random_state=42, shuffle=True)

for train_index, test_index in skfolds.split(X_train, y_train_5_book):
    clone_clf = clone(sgd_clf)
    X_train_folds = X_train[train_index]
    y_train_folds = y_train_5_book[train_index]
    X_test_folds = X_train[test_index]
    y_test_folds = y_train_5_book[test_index]
    clone_clf.fit(X_train_folds, y_train_folds)
    y_pred = clone_clf.predict(X_test_folds)
    n_correct = sum(y_pred == y_test_folds)
    print(n_correct/len(y_pred))

0.9669
0.91625
0.96785

from sklearn.model_selection import cross_val_score
cross_val_score(sgd_clf, X_train, y_train_5_book, cv=3,
scoring="accuracy")

array([0.95035, 0.96035, 0.9604 ])

```

Accuracy is not always the best measure for a classification model. Precision and recall and f1 score is considered to be more appropriate measure to measure the performance of a classification model.

```

from sklearn.model_selection import cross_val_predict
y_train_pred = cross_val_predict(sgd_clf, X_train, y_train_5_book,
cv=3)

```

A confusion matrix of a binary classifier has 4 columns true negative, false positive, false negative and true positive. Confusion matrix gives a good measure for classification models.

```

from sklearn.metrics import confusion_matrix
confusion_matrix(y_train_5_book, y_train_pred)

array([[53892,   687],
       [ 1891,  3530]])

```

precision is $tp/(tp + tn)$ and recall is $tp/(tp + fn)$

precision is the measure of how correct the model is and recall is the measure of how many wrong predictions the model is making.

when you are building such a model where you cannot accept any wrong being classified as right you choose a model which has high precision

but when it is important for you to identify all the correct ones even if your model is sometimes terming wrong as right then you choose a model which has high recall

```
from sklearn.metrics import precision_score, recall_score
precision = precision_score(y_train_5_book, y_train_pred)
recall = recall_score(y_train_5_book, y_train_pred)
print(f"Precision: {precision}\nRecall: {recall}")
```

```
Precision: 0.8370879772350012
Recall: 0.6511713705958311
```

f1 score is the harmonic mean of precision score and recall score

```
from sklearn.metrics import f1_score
f1 = f1_score(y_train_5_book, y_train_pred)
print(f"The f1 score is: {f1}")
```

```
The f1 score is: 0.7325171197343847
```

decision function gives the threshold which helps the model to predict what value the digit is

```
decision_scores = sgd_clf.decision_function(X_train[:15])
for i, j in zip(decision_scores, y_train_5_book[:15]):
    print(f"{i}: {j}")
```

```
2164.2203023926586: True
-5897.373593538878: False
-13489.148057785078: False
-4623.845159432227: False
-6873.052484528889: False
-3908.2757629071402: False
-4851.942239699033: False
-4988.524077546987: False
-2226.7813286640758: False
-4081.3369283115107: False
-5711.4778968723085: False
4742.528131575118: True
-10322.619700870204: False
-5028.508042401304: False
-2774.489337011428: False
```

```
decision_scores = cross_val_predict(sgd_clf, X_train, y_train_5_book,
method="decision_function", cv=3)
y_scores = decision_scores.copy()
```

decision function can be used to calculate the precision_recall curve

```
from sklearn.metrics import precision_recall_curve
precisions, recalls, thresholds =
precision_recall_curve(y_train_5_book, y_scores)
```

```
thresholds.shape
```

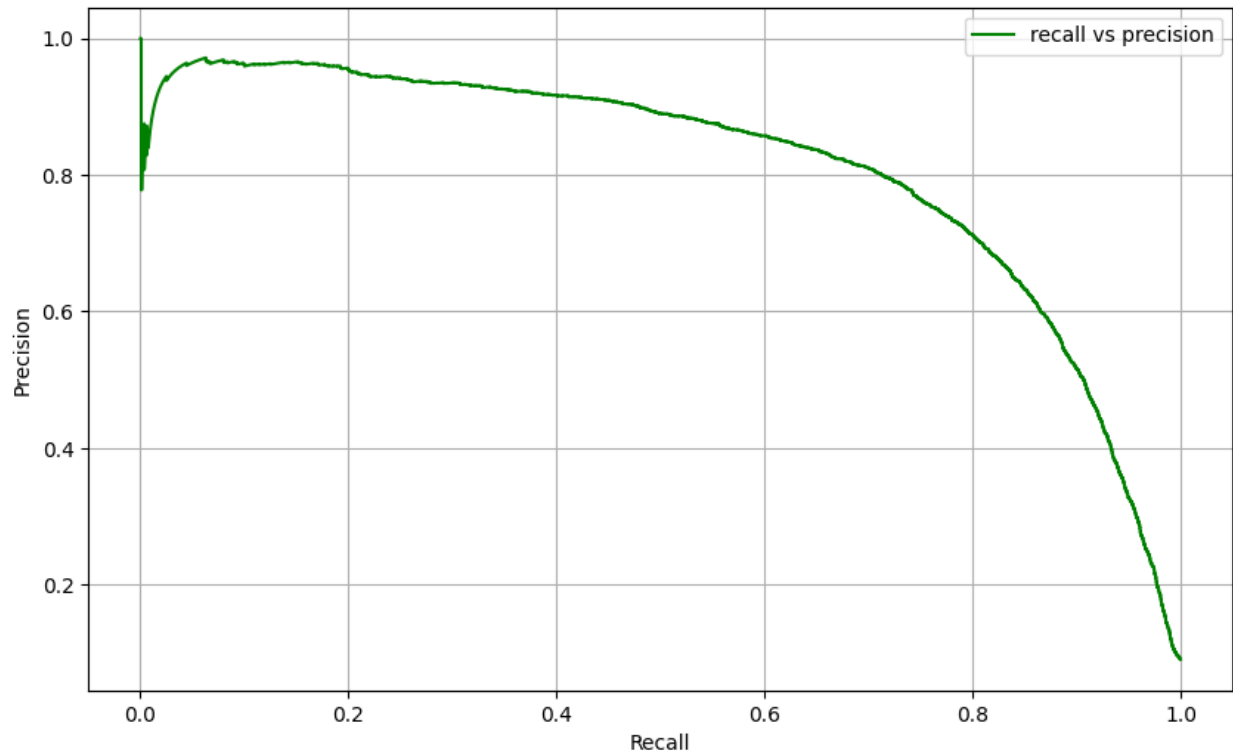
```
(60000,)
```

```
print(f"The dimentions of \nprecision is: {precisions.shape}\nrecalls
is: {recalls.shape}\nthreshold is:{thresholds.shape}")
```

```
The dimentions of
precision is: (60001,)
recalls is: (60001,)
threshold is:(60000,)
```

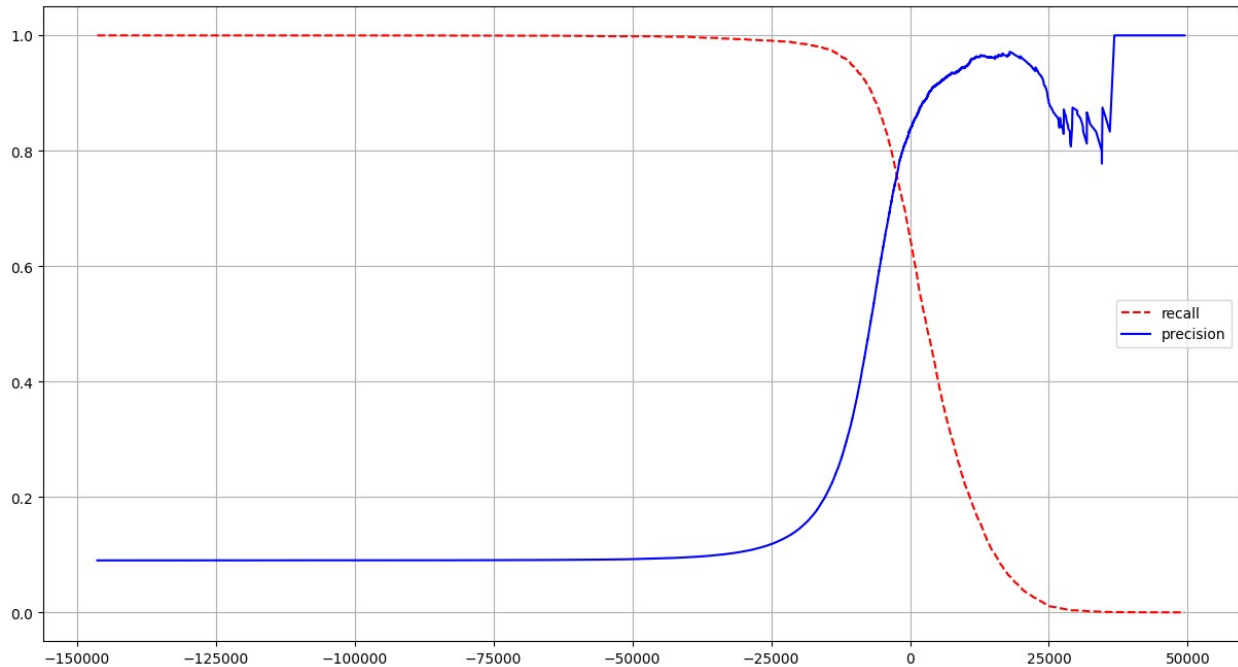
drawing the recall vs precision curve to know the correct balance between the recall and precision

```
plt.figure(figsize=(10, 6))
plt.plot(recalls[:recalls.shape[0]-1],
precisions[:precisions.shape[0]-1], "g-", label="recall vs
precision",)
plt.xlabel("Recall")
plt.ylabel("Precision")
plt.legend()
plt.grid(True)
plt.show()
```



Plotting recall and precision against threshold to find the tradeoff suitable for our model

```
plt.figure(figsize=(15, 8))
plt.plot(thresholds, recalls[:-1], "r--", label="recall")
plt.plot(thresholds, precisions[:-1], "b", label="precision")
plt.grid(True)
plt.legend(loc=7)
plt.show()
```

```
threshold_90_precision = thresholds[np.argmax(precisions>=0.90)]
threshold_90_precision

np.float64(3370.0194991439557)

threshold_90_recall = thresholds[np.argmin(recalls>=0.90)]
threshold_90_recall

np.float64(-6861.032537940254)

y_train_pred_precision_90 = (y_scores >= threshold_90_precision)
newPrecision = precision_score(y_train_5_book,
y_train_pred_precision_90)
newRecall = recall_score(y_train_5_book, y_train_pred_precision_90)
print(f"The new precision is {newPrecision}\nThe new recall is
{newRecall}")
```

```
The new precision is 0.9000345901072293
The new recall is 0.4799852425751706
```

```
def setPrecision(prec_val, thresholds, precisions, scores, label) ->
None:
    thresholdPrecision = thresholds[np.argmax(precisions >= prec_val)]
    print(f"For {prec_val} precision:\nThe new threshold is
{thresholdPrecision}")
    y_train_new_precision = (scores >= thresholdPrecision)
    newPrec = precision_score(y_train_5_book, y_train_new_precision)
    newRec = recall_score(y_train_5_book, y_train_new_precision)
    print(f"The new precision is {newPrec}\nThe new recall is
{newRec}")
```

```
setPrecision(0.90, thresholds, precisions, y_scores, y_train_5_book)
```

For 0.9 precision:

The new threshold is 3370.0194991439557

The new precision is 0.9000345901072293

The new recall is 0.4799852425751706

```
setPrecision(0.75, thresholds, precisions, y_scores, y_train_5_book)
```

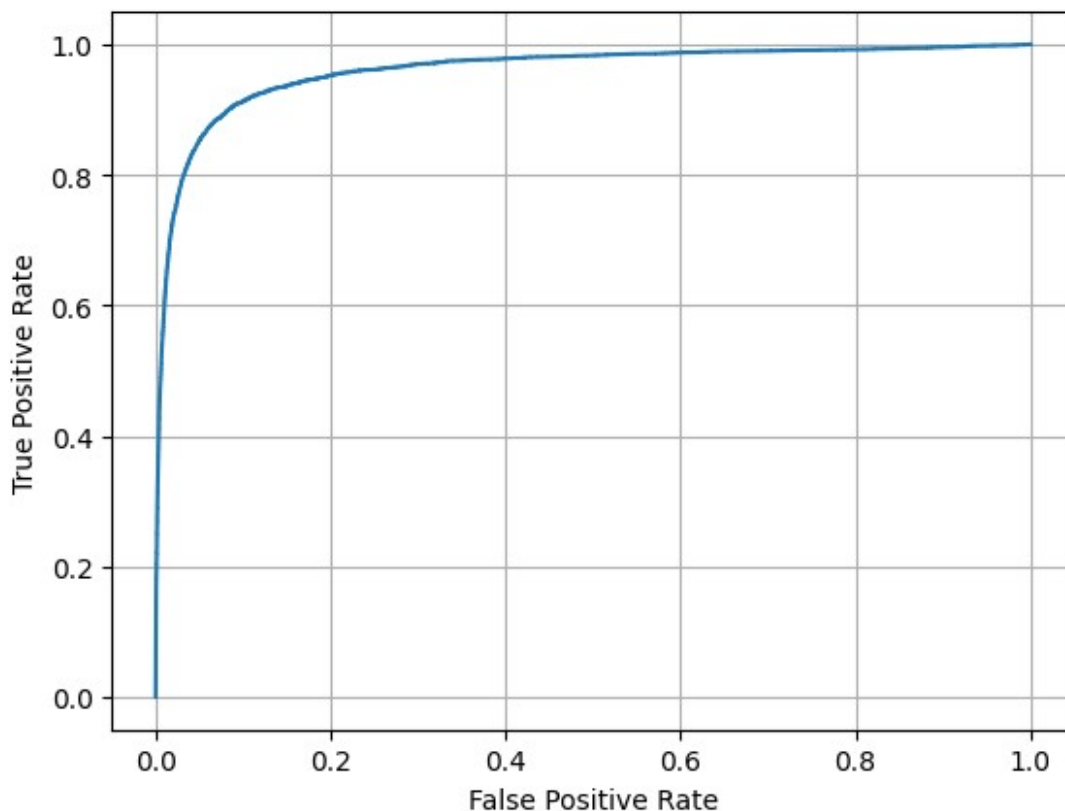
For 0.75 precision:

The new threshold is -2551.6670906292175

The new precision is 0.75

The new recall is 0.7659103486441616

```
from sklearn.metrics import roc_curve  
fpr, tpr, thresholds = roc_curve(y_train_5_book, y_scores)  
plt.plot(fpr, tpr)  
plt.grid(True)  
plt.xlabel("False Positive Rate")  
plt.ylabel("True Positive Rate")  
plt.show()
```



```
from sklearn.metrics import roc_auc_score
roc_auc_scores = roc_auc_score(y_train_5_book, y_scores)
print(f"The roc area under curve score is: {roc_auc_scores}")
```

The roc area under curve score is: 0.9604938554008616

```
from sklearn.ensemble import RandomForestClassifier
rfc_classifier = RandomForestClassifier(random_state = 42)
cross_val_score(rfc_classifier, X_train, y_train_5_book, cv=3,
scoring="accuracy")
```

```
array([0.98885, 0.9863 , 0.98635])
```

```
forest_d_scores = cross_val_predict(rfc_classifier, X_train,
y_train_5_book, cv=3, method="predict_proba")
```

```
forest_d_scores.shape
```

```
(60000, 2)
```

```
roc_auc_rfc_scores = roc_auc_score(y_train_5_book, forest_d_scores[:,
1])
```

```
print(f"The roc score for random forest is: {roc_auc_rfc_scores}")
```

The roc score for random forest is: 0.9983436731328145

```
forest_pred_scores = cross_val_predict(rfc_classifier, X_train,
y_train_5_book, cv=3)
confusion_matrix(y_train_5_book, forest_pred_scores)
```

```
array([[54534,    45],
       [   725,  4696]])
```

```
precision_score(y_train_5_book, forest_pred_scores)
```

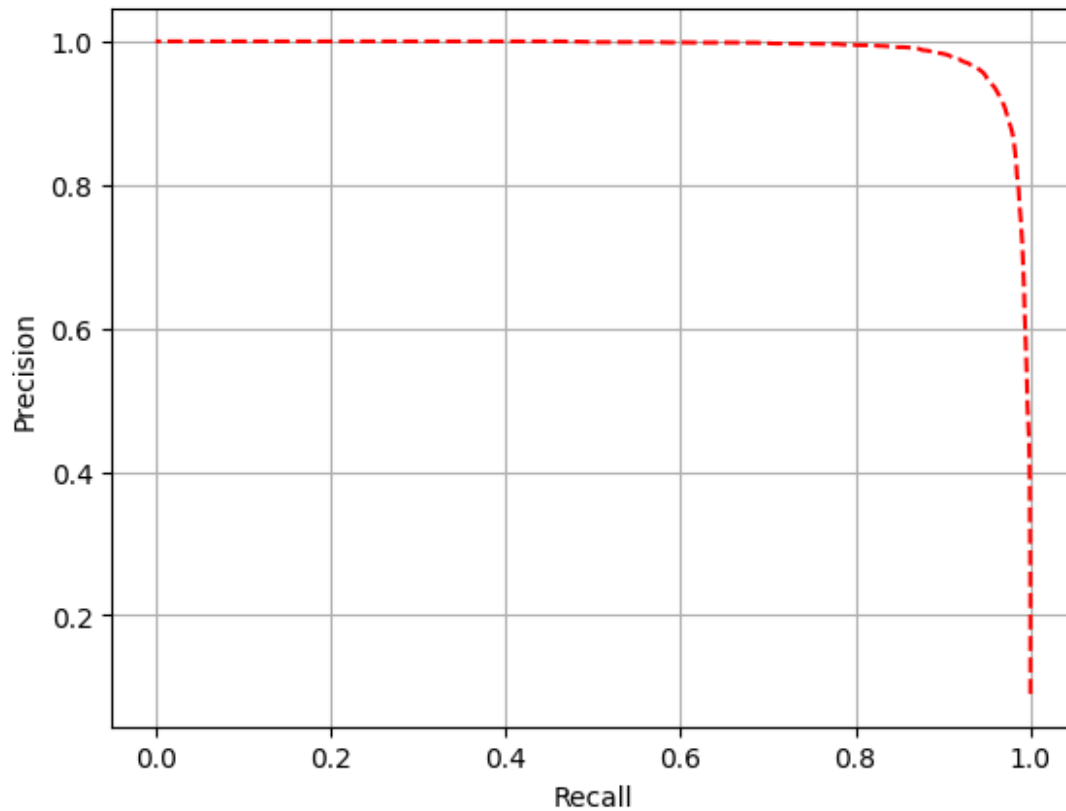
```
0.9905083315756169
```

```
recall_score(y_train_5_book, forest_pred_scores)
```

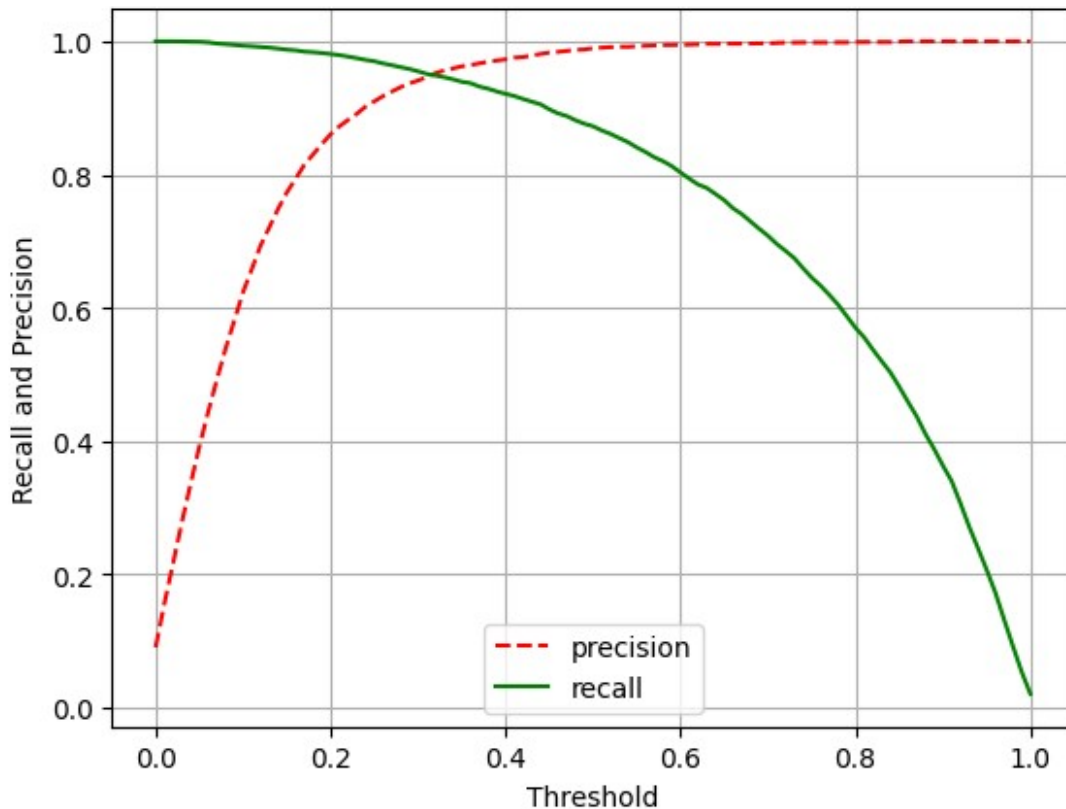
```
0.8662608374838591
```

```
precision_forest, recall_forest, threshold_forest =
precision_recall_curve(y_train_5_book, forest_d_scores[:,1])
```

```
plt.plot(recall_forest, precision_forest, "r--", label="recall vs
precision")
plt.xlabel("Recall")
plt.ylabel("Precision")
plt.grid(True)
plt.show()
```



```
plt.plot(threshold_forest, precision_forest[:-1], "r--",  
label="precision")  
plt.plot(threshold_forest, recall_forest[:-1], "g", label="recall")  
plt.xlabel("Threshold")  
plt.ylabel("Recall and Precision")  
plt.legend(loc=8)  
plt.grid(True)  
plt.show()
```



```

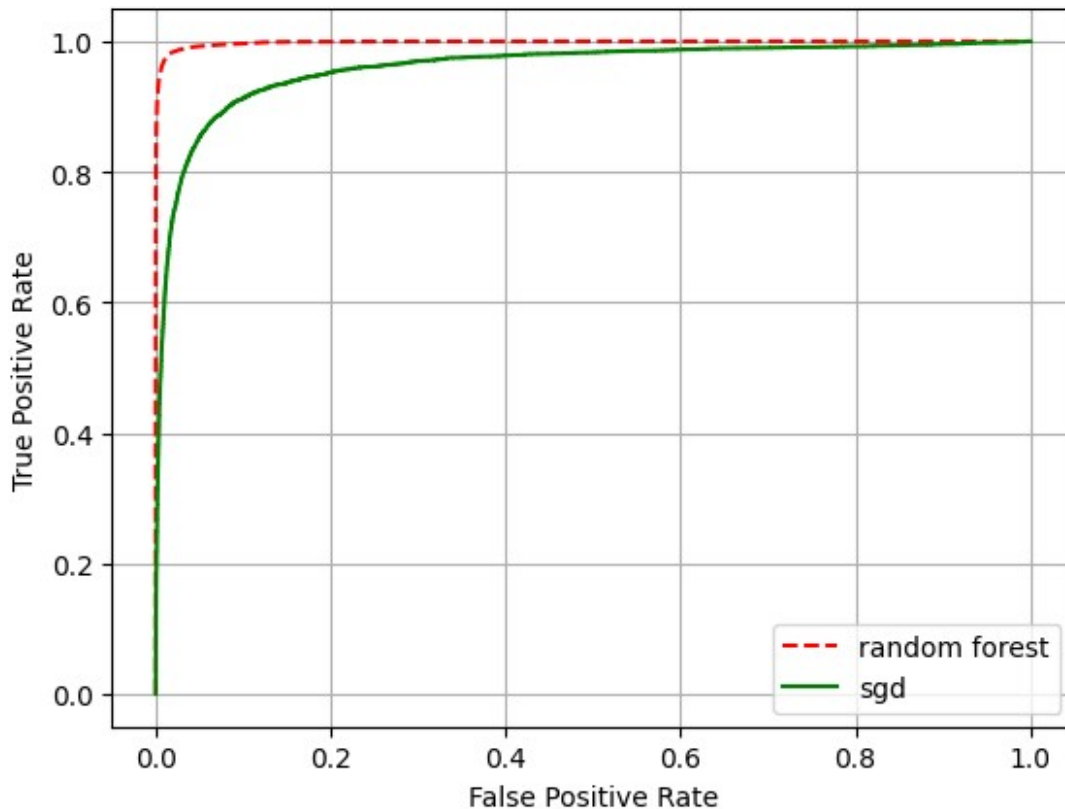
setPrecision(0.95, threshold_forest, precision_forest,
forest_d_scores[:, 1], y_train_5_book)

For 0.95 precision:
The new threshold is 0.32
The new precision is 0.9522310683206814
The new recall is 0.9487179487179487

fpr_forest, tpr_forest, threshold_rfc = roc_curve(y_train_5_book,
forest_d_scores[:, 1])

plt.plot(fpr_forest, tpr_forest, "r--", label="random forest")
plt.plot(fpr, tpr, "g", label="sgd",)
plt.grid(True)
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.legend(loc="lower right")
plt.show()

```



```
from sklearn.model_selection import GridSearchCV
try:
    best_sgd_model = joblib.load("best_sgd_model_binary.pkl")
    print("Saved model found\nImported from saved model")
except Exception as e:
    param_grid = [
        {
            "loss": ['hinge', 'squared_hinge'],
            "max_iter": [200, 500, 1000, 2000, 3000],
            "n_jobs": [-1],
            "learning_rate": ['optimal'],
        }
    ]
    sgd_for_grid = SGDClassifier()
    grid_search = GridSearchCV(sgd_for_grid, param_grid,
                              scoring="accuracy",
                              n_jobs=-1, cv=5,
                              return_train_score=True)
    grid_search.fit(X_train, y_train_5_book)
    best_sgd_model = grid_search.best_estimator_
    joblib.dump(best_sgd_model, "best_sgd_model_binary.pkl")
```

Saved model found
Imported from saved model

```
best_sgd_pred = best_sgd_model.predict(X_train)
precision_score_best_sgd = precision_score(y_train_5_book,
best_sgd_pred)
recall_score_best_sgd = recall_score(y_train_5_book, best_sgd_pred)
print(f"The precision is: {precision_score_best_sgd}\nThe recall
score: {recall_score_best_sgd}")
```

The precision is: 0.9403327596098681
The recall score: 0.6046854823833241

```
best_sgd_decision_scores = best_sgd_model.decision_function(X_train)
roc_score_best_sgd = roc_auc_score(y_train_5_book,
best_sgd_decision_scores)
print(f"The roc auc score is {roc_score_best_sgd}")
```

The roc auc score is 0.9729704720805338

```
try:
    sgd_full_data = joblib.load("ovr_sgd_clf.pkl")
    print("Saved model found\nImported from saved model")
except Exception as e:
    sgd_full_data = SGDClassifier()
    sgd_full_data.fit(X_train, y_train)
    joblib.dump(sgd_full_data, "ovr_sgd_clf.pkl")
```

Saved model found
Imported from saved model

```
sgd_full_data.predict([raw_data[11], some_digit])
```

```
array([5, 3], dtype=uint8)
```

```
sgd_full_data.decision_function([raw_data[11]])
```

```
array([[ -24979.72160379, -10472.2614079 , -4057.98638728,
        -11284.4487607 , -7330.4273682 ,  3276.71960605,
        -16752.3031591 , -11097.09936133, -503.07122549,
        -10474.04110308]])
```

```
from sklearn.multiclass import OneVsOneClassifier
try:
    ovo_sgd_clf = joblib.load("ovo_sgd_clf.pkl")
    print("Saved model found\nImported from saved model")
except Exception as e:
    ovo_sgd_clf = OneVsOneClassifier(SGDClassifier(random_state=42,
n_jobs=-1))
    ovo_sgd_clf.fit(X_train, y_train)
    joblib.dump(ovo_sgd_clf, "ovo_sgd_clf.pkl")
```

Saved model found
Imported from saved model

```

ovo_sgd_clf.predict([some_digit, raw_data[11]])
array([5, 5], dtype=uint8)
try:
    random_full_forest = joblib.load("random_full_dataset.pkl")
    print("Saved model found\nImported from saved model")
except Exception as e:
    random_full_forest = RandomForestClassifier(random_state=42,
n_jobs=-1)
    random_full_forest.fit(X_train, y_train)
    joblib.dump(random_full_forest, "random_full_dataset.pkl")

Saved model found
Imported from saved model

random_full_forest.predict([some_digit, raw_data[11]])
array([5, 5], dtype=uint8)

random_full_forest.predict_proba([some_digit, raw_data[11]])
array([[0.   , 0.   , 0.01, 0.08, 0.   , 0.9 , 0.   , 0.   , 0.   , 0.01],
       [0.01, 0.05, 0.02, 0.   , 0.   , 0.9 , 0.   , 0.   , 0.01, 0.01]])

ovo_sgd_cross_scores = cross_val_score(ovo_sgd_clf, X_train, y_train,
scoring="accuracy", n_jobs=-1)

ovo_sgd_cross_scores
array([0.92025   , 0.91641667, 0.91575   , 0.89841667, 0.91225   ])

ovr_sgd_cross_val_scores = cross_val_score(sgd_full_data, X_train,
y_train, scoring="accuracy", n_jobs=-1)

ovr_sgd_cross_val_scores
array([0.86758333, 0.88816667, 0.85033333, 0.8725   , 0.88891667])

random_forest_full_cross_val_scores =
cross_val_score(random_full_forest, X_train, y_train,
scoring="accuracy", n_jobs=-1)

random_forest_full_cross_val_scores
array([0.96808333, 0.96566667, 0.96541667, 0.962   , 0.97108333])

from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train.astype(np.float64))
random_forest_full_scaled_cross_val_scores =
cross_val_score(random_full_forest, X_train_scaled, y_train,

```



```

scoring="accuracy", n_jobs=-1)
print(random_forest_full_scaled_cross_val_scores)

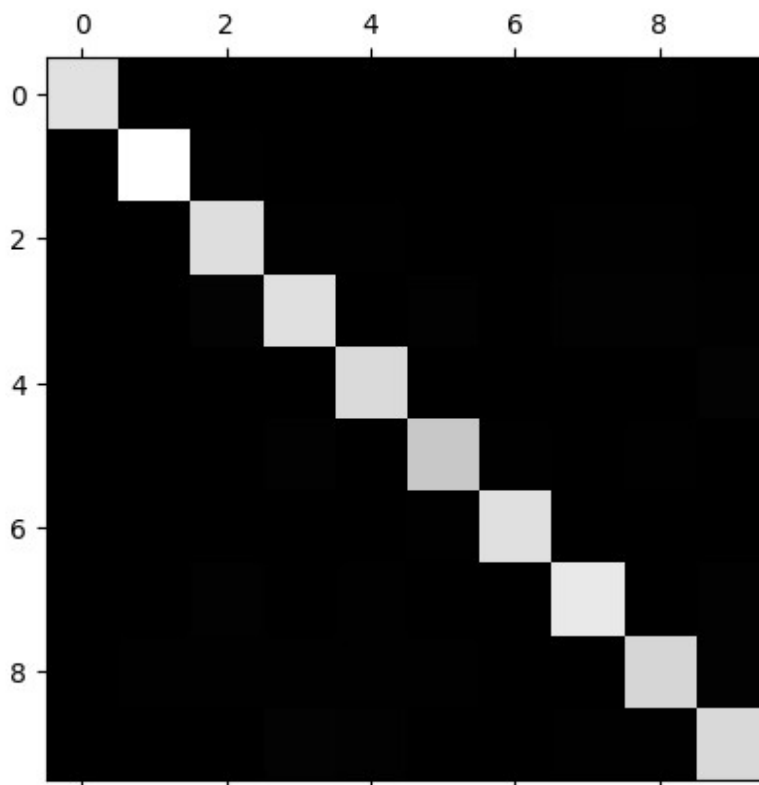
[0.96816667 0.96575    0.96591667 0.962    0.971    ]

y_train_pred_r_forest_cross = cross_val_predict(random_full_forest,
X_train_scaled, y_train)
conf_mx = confusion_matrix(y_train, y_train_pred_r_forest_cross)
conf_mx

array([[5846,    2,    7,    2,    4,    6,   19,    1,   32,    4],
       [    1, 6637,   32,   16,   14,    4,    7,   15,   10,    6],
       [   24,   11, 5756,   30,   31,    2,   19,   39,   39,    7],
       [    8,    6,   82, 5819,    1,   65,    8,   53,   61,   28],
       [   12,   12,   11,    0, 5671,    0,   20,   11,   17,   88],
       [   25,    7,   11,   58,    9, 5210,   41,    5,   32,   23],
       [   25,   11,    1,    0,    8,   32, 5818,    0,   23,    0],
       [    4,   22,   64,    6,   42,    1,    1, 6044,   13,   68],
       [   11,   28,   35,   51,   26,   49,   25,    6, 5554,   66],
       [   24,   11,   16,   79,   69,   18,    4,   51,   38, 5639]])

plt.matshow(conf_mx, cmap=plt.cm.gray)
plt.show()

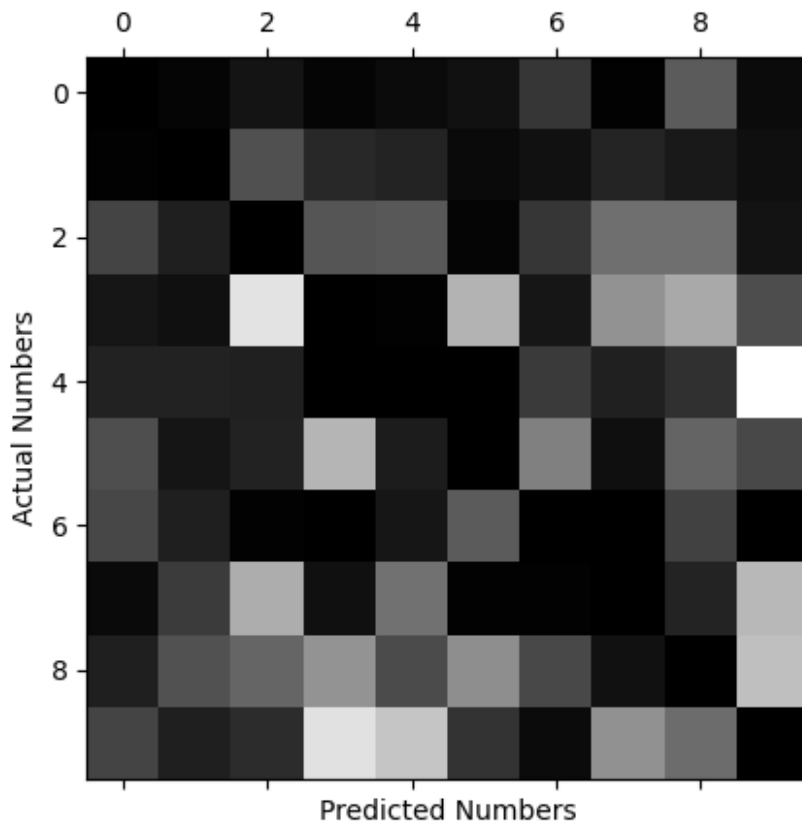
```



Find the row sum to see the rate of good detection

```
row_sum = conf_mx.sum(axis=1, keepdims=True)
norm_conf_mx = conf_mx / row_sum

np.fill_diagonal(norm_conf_mx, 0)
plt.matshow(norm_conf_mx, cmap=plt.cm.gray)
plt.xlabel("Predicted Numbers")
plt.ylabel("Actual Numbers")
plt.show()
```



Let's train a multiclass classification system which identifies if a digit is even or odd

```
from sklearn.neighbors import KNeighborsClassifier
y_train_even = (y_train % 2 == 0)
y_train_odd = (y_train % 2 != 0)
y_even_odd = np.c_[y_train_even, y_train_odd]
try:
    knn_clf = joblib.load("knn_even_odd.pkl")
    print("Imported from saved models")
except Exception as e:
    knn_clf = KNeighborsClassifier(n_jobs=-1, n_neighbors=10)
    knn_clf.fit(X_train, y_even_odd)
    joblib.dump(knn_clf, "knn_even_odd.pkl")
```

```
Imported from saved models
random_full_forest.predict([some_digit, raw_data[20]])
array([5, 4], dtype=uint8)
knn_clf.predict([some_digit, raw_data[20]])
array([[False,  True],
       [ True, False]])
```

Evaluating the multiclass model based on f1 score

```
knn_cross_val_predict = cross_val_predict(knn_clf, X_train,
y_even_odd)
f1_multiclass_cross = f1_score(y_even_odd, knn_cross_val_predict,
average="macro")
print(f"{f1_multiclass_cross} is the f1 score for multiclass model.")
0.9792989977679984 is the f1 score for multiclass model.

f1_multiclass_cross_weighted = f1_score(y_even_odd,
knn_cross_val_predict, average="weighted")
print(f"{f1_multiclass_cross_weighted} is the f1 score for multiclass
model.(average set to weighted)")
0.979333825253079 is the f1 score for multiclass model.(average set to
weighted)

knn_predict = knn_clf.predict(X_train)
f1_multiclass = f1_score(y_even_odd, knn_predict, average="macro")
print(f"{f1_multiclass} is the f1 score for multiclass model(without
cross val).")
0.9853803196663053 is the f1 score for multiclass model(without cross
val).

f1_multiclass_weighted = f1_score(y_even_odd, knn_predict,
average="weighted")
print(f"{f1_multiclass_weighted} is the f1 score for multiclass model.
(average set to weighted)")
0.9854026471568955 is the f1 score for multiclass model.(average set
to weighted)
```

Multiclass Classification

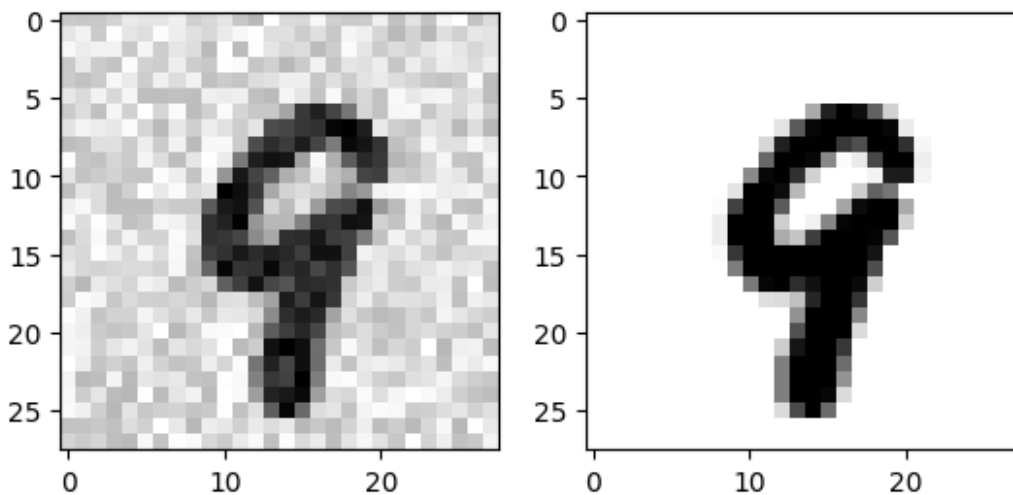
clearing up images with noise in them with the help of multiclass classification

- new train set - we will add external noise to the train dataset
- new train label - the old train set which is the set of clear image
- new test set - we will add noise to the old test set

- new test label - the old test set. **This has no connection with the prediction of numbers**

```
noise = np.random.randint(0, 100, X_train.shape)
X_train_mod = X_train + noise
noise = np.random.randint(0, 100, X_test.shape)
X_test_mod = X_test + noise

%matplotlib inline
import matplotlib as mpl
import matplotlib.pyplot as plt
some_digit = X_train_mod[6969]
some_digit_image = some_digit.reshape(28, 28)
plt.subplot(1, 2, 1)
plt.imshow(some_digit_image, cmap = mpl.cm.binary,
interpolation="nearest")
plt.axis("on")
plt.grid(False)
some_digit = X_train[6969]
some_digit_image = some_digit.reshape(28, 28)
plt.subplot(1, 2, 2)
plt.imshow(some_digit_image, cmap = mpl.cm.binary,
interpolation="nearest")
plt.axis("on")
plt.grid(False)
plt.show()
```



```
print(y_train[6969])
9
try:
    knn_multiclass=joblib.load("knn_multiclass.pkl")
except Exception as e:
```

```

knn_multiclass = KNeighborsClassifier(n_neighbors=10, n_jobs=-1)
knn_multiclass.fit(X_train_mod, X_train)
joblib.dump(knn_multiclass, "knn_multiclass.pkl")

pred_num = knn_multiclass.predict([X_train_mod[6969]])
some_digit = pred_num
some_digit_image = some_digit.reshape(28, 28)
plt.imshow(some_digit_image, cmap = mpl.cm.binary,
            interpolation="nearest")
plt.axis("on")
plt.grid(False)
plt.show()

```

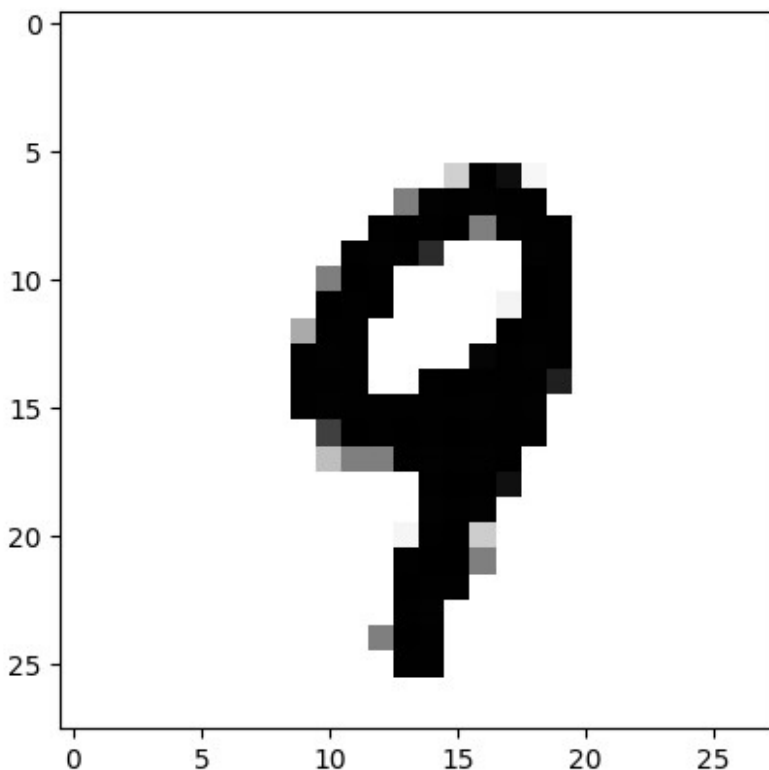


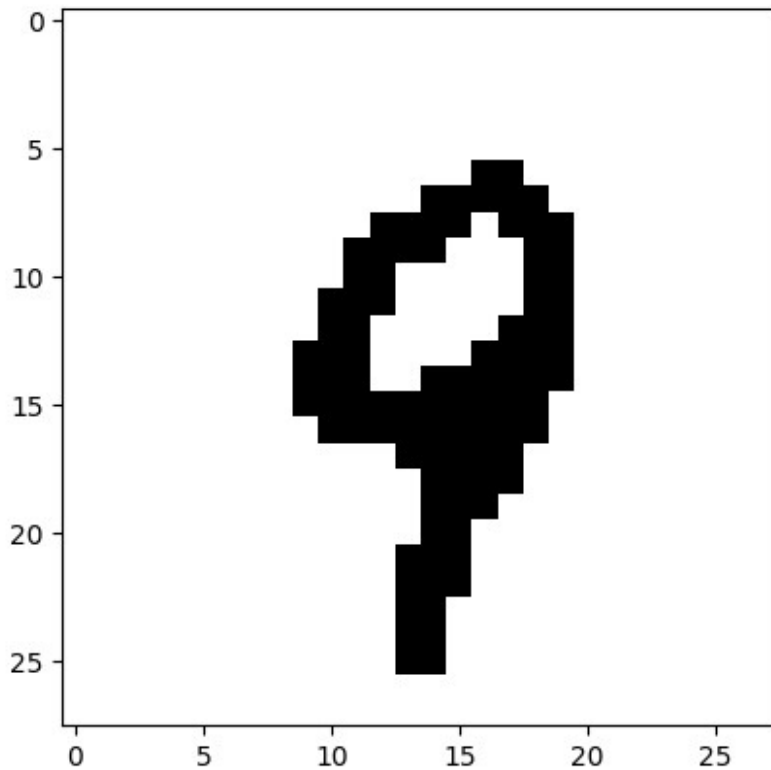
image cleaning without using classification algorithms

```

trail_clean = some_digit_image.copy()
for i in range(28):
    for j in range(28):
        if(trail_clean[i][j] < 180):
            trail_clean[i][j] = 0
        else:
            trail_clean[i][j] = 255

plt.imshow(trail_clean, cmap = mpl.cm.binary, interpolation="nearest")
plt.axis("on")
plt.grid(False)

```



Testing the random forest model on built on the full dataset on various scores

```
def judge_scores(model, data, labels):
    random_x_train_pred = model.predict(data)
    p_score = precision_score(labels, random_x_train_pred,
    average="weighted")
    r_score = recall_score(labels, random_x_train_pred,
    average="weighted")
    f_score = f1_score(labels, random_x_train_pred,
    average="weighted")
    print(f"The precision score is : {p_score}\nThe recall score is:
    {r_score}\nThe f1 score is: {f_score}\n")

    random_x_train_pred = cross_val_predict(model, data, labels,
    method="predict")
    p_score = precision_score(labels, random_x_train_pred,
    average="weighted")
    r_score = recall_score(labels, random_x_train_pred,
    average="weighted")
    f_score = f1_score(labels, random_x_train_pred,
    average="weighted")
    print(f"The precision score is : {p_score}\nThe recall score is:
    {r_score}\nThe f1 score is: {f_score}")
```

```
judge_scores(random_full_forest, X_train, y_train)
```

```
The precision score is : 1.0
```

```
The recall score is: 1.0
```

```
The f1 score is: 1.0
```

```
The precision score is : 0.9664253796396532
```

```
The recall score is: 0.96645
```

```
The f1 score is: 0.9664247196315459
```

Calculating the scores for the test set

```
judge_scores(random_full_forest, X_test, y_test)
```

```
The precision score is : 0.970495753186874
```

```
The recall score is: 0.9705
```

```
The f1 score is: 0.9704722695987773
```

```
The precision score is : 0.9466470668021917
```

```
The recall score is: 0.9467
```

```
The f1 score is: 0.9465894841016514
```

```
try:
```

```
    best_random_full = joblib.load("best_random_full.pkl")
```

```
    print("Exported from saved models")
```

```
except Exception as e:
```

```
    param_grid = {
```

```
        "n_estimators": [750, 1000],
```

```
        "max_depth": [None,],
```

```
        "min_samples_split": [2, 100],
```

```
        "random_state": [42],
```

```
        "n_jobs": [-1],
```

```
    }
```

```
    random_full_clf = RandomForestClassifier()
```

```
    grid_search = GridSearchCV(random_full_clf, param_grid,
```

```
                               scoring="accuracy",
```

```
                               n_jobs=-1, cv=5,
```

```
                               return_train_score=True)
```

```
    grid_search.fit(X_train, y_train)
```

```
    best_random_full = grid_search.best_estimator_
```

```
    joblib.dump(best_random_full, "best_random_full.pkl")
```

Exported from saved models

```
judge_scores(best_random_full, X_train, y_train)
```

```
The precision score is : 1.0
```

```
The recall score is: 1.0
```

```
The f1 score is: 1.0
```

The precision score is : 0.9688930283786341

The recall score is: 0.9689

The f1 score is: 0.9688851849099264

```
judge_scores(best_random_full, X_test, y_test)
```

The precision score is : 0.9718046138942451

The recall score is: 0.9718

The f1 score is: 0.9717869971511677

The precision score is : 0.9497943684019255

The recall score is: 0.9498

The f1 score is: 0.9497140377779512

```
# from sklearn.neighbors import KNeighborsClassifier
# try:
#     best_knn_full = joblib.load("best_knn_full.pkl")
#     print("Exported from saved models")
# except Exception as e:
#     param_grid_knn = {
#         "n_neighbors":[1, 2, 3, 5],
#         "weights": ['uniform', 'distance'],
#         # "n_jobs": [-1],
#         "leaf_size":[5],
#     }
#     knn_full_clf = KNeighborsClassifier()
#     grid_search_knn = GridSearchCV(knn_full_clf, param_grid_knn,
#                                     scoring="accuracy",
#                                     n_jobs=-1, cv=5,
#                                     return_train_score=True)
#     grid_search_knn.fit(X_train, y_train)
#     best_random_full = grid_search.best_estimator_
#     joblib.dump(best_knn_full, "best_knn_full.pkl")
```

Postpone the training of knn model due to lack of system resource

Function for shifting the image by one pixel in the desired direction

```
def shift_image(np_array, direction="left"):
    import copy
    array = copy.deepcopy(np_array)
    if direction == "left":
        for i in range(len(array)):
            for j in range(1, len(array[i])):
                array[i][j-1] = array[i][j]
    elif direction == "right":
        for i in range(len(array)-1, -1, -1):
            for j in range(len(array[i]) - 1, 0, -1):
                array[i][j] = array[i][j-1]
    elif direction == "down":
```



```

    temp = []
    for i in range(len(array)):
        temp.append(array[i].reshape(28, 28))
    temp = np.array(temp)
    for i in range(len(temp)):
        for j in range(28):
            for k in range(27, 0, -1):
                temp[i][k][j] = temp[i][k-1][j]
        return temp
elif direction == "up":
    temp = []
    for i in range(len(array)):
        temp.append(array[i].reshape(28, 28))
    temp = np.array(temp)
    for i in range(len(temp)):
        for j in range(28):
            for k in range(27):
                temp[i][k][j] = temp[i][k+1][j]
        return temp
return array

new_array_right = shift_image(X_train, direction="right")
new_array_left = shift_image(X_train, direction="left")
new_array_up = shift_image(X_train, direction="up")
new_array_down = shift_image(X_train, direction="down")

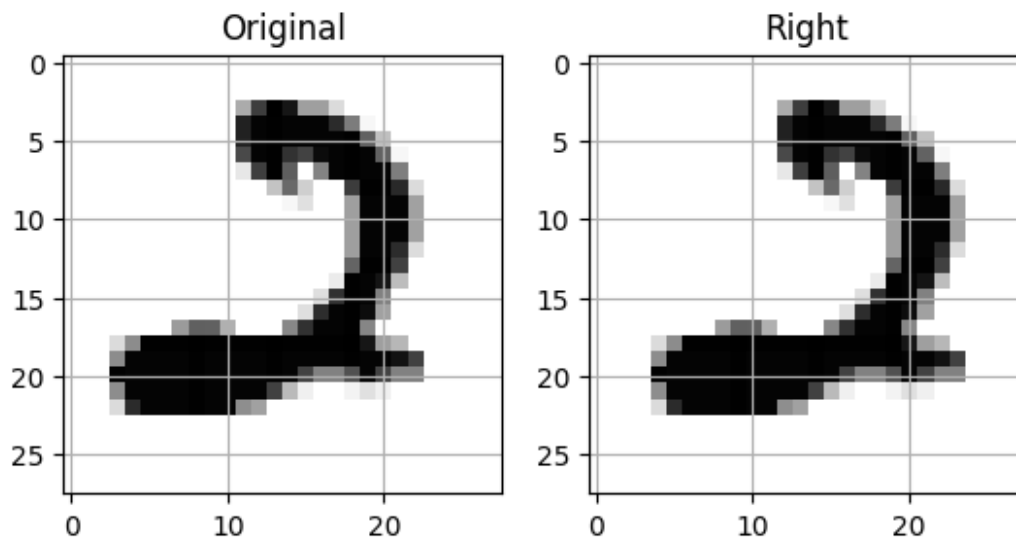
def show_image(array, grid:bool=False) -> None:
    image = array.reshape(28, 28)
    plt.imshow(image, cmap=matplotlib.cm.binary, interpolation="nearest")
    plt.axis("on")
    plt.grid(grid)

fig1 = plt.subplot(1, 2, 1)
fig1.set_title("Original")
show_image(X_train[1010], True)

fig2 = plt.subplot(1, 2, 2)
fig2.set_title("Right")
show_image(new_array_right[1010], True)

plt.show()

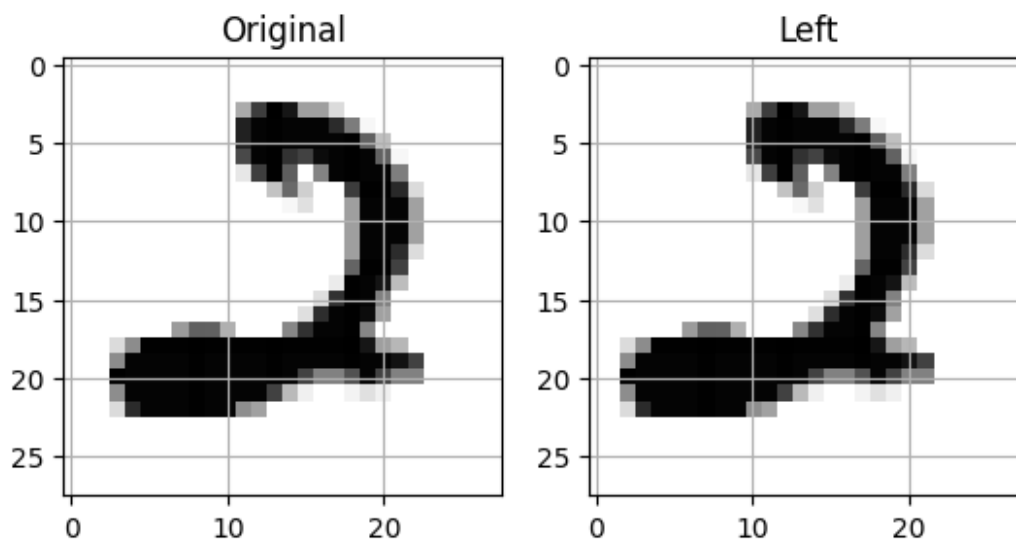
```



```
fig3 = plt.subplot(1, 2, 1)
fig3.set_title("Original")
show_image(X_train[1010], True)

fig4 = plt.subplot(1, 2, 2)
fig4.set_title("Left")
show_image(new_array_left[1010], True)

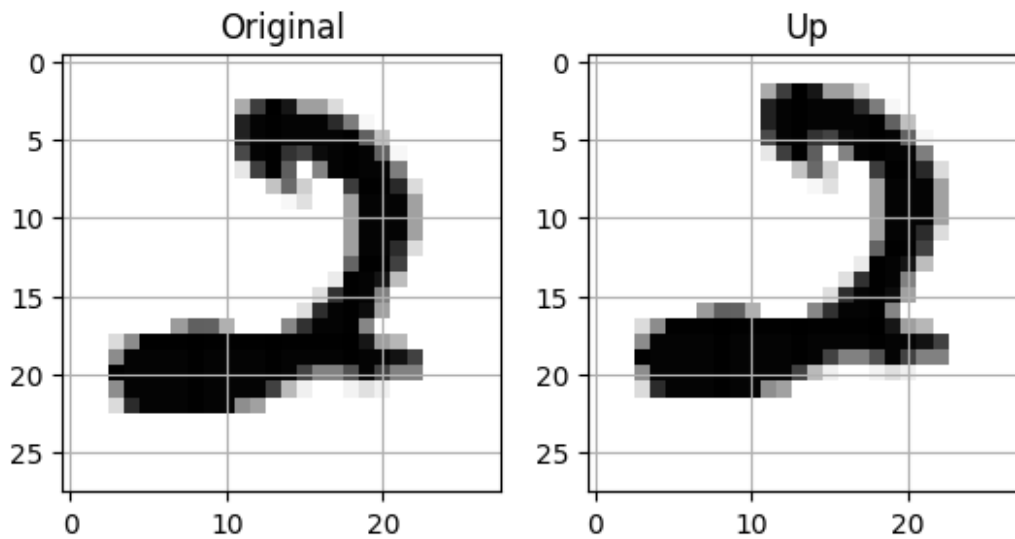
plt.show()
```



```
fig5 = plt.subplot(1, 2, 1)
fig5.set_title("Original")
show_image(X_train[1010], True)
```

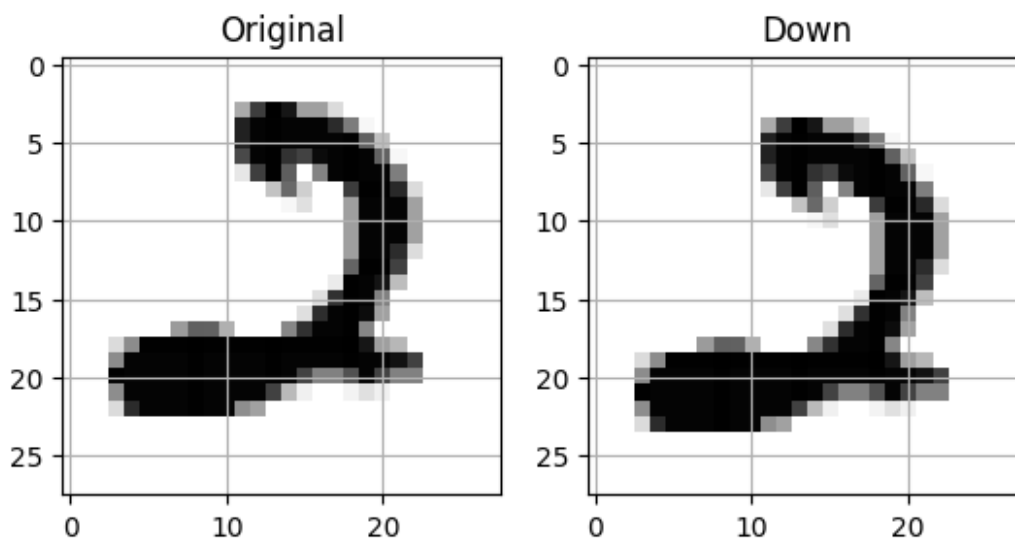
```
fig6 = plt.subplot(1, 2, 2)
fig6.set_title("Up")
show_image(new_array_up[1010], True)

plt.show()
```



```
fig7 = plt.subplot(1, 2, 1)
fig7.set_title("Original")
show_image(X_train[1010], True)

fig8 = plt.subplot(1, 2, 2)
fig8.set_title("Down")
show_image(new_array_down[1010], True)
```



```

new_array_down = new_array_down.reshape(60000, 784)
new_array_up = new_array_up.reshape(60000, 784)

X_train_copy = copy.deepcopy(X_train)
X_train_copy = np.concatenate([X_train_copy, new_array_down,
new_array_left, new_array_right, new_array_up])

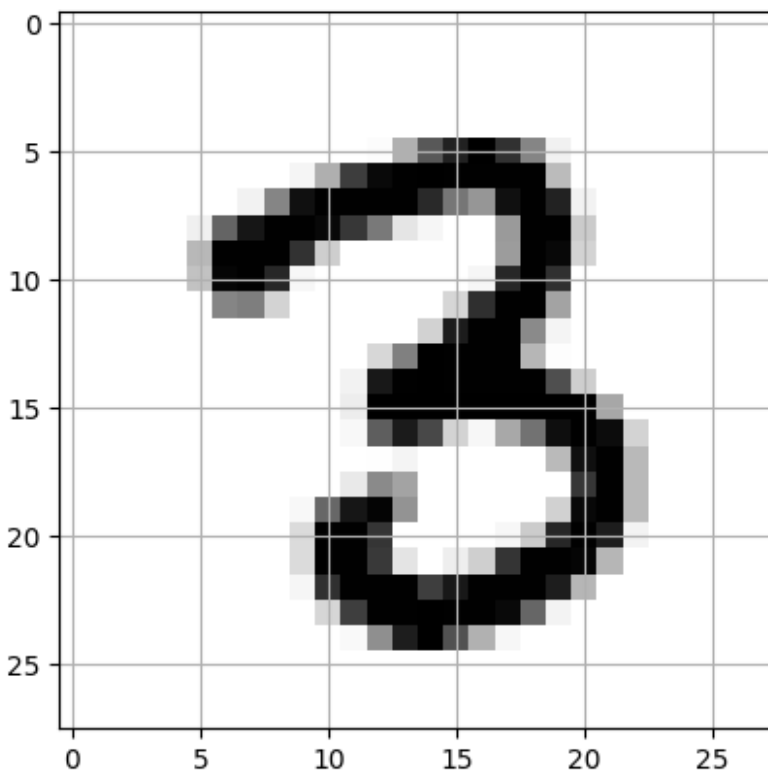
y_train_copy = copy.deepcopy(y_train)
y_train_copy = np.concatenate([y_train, y_train, y_train, y_train,
y_train])

print(f"labels shpae: {y_train_copy.shape} data shape:
{X_train_copy.shape}")

labels shpae: (300000,) data shape: (300000, 784)

show_image(X_train_copy[234560], True)
plt.show()
print(y_train_copy[234560])

```



3

```

judge_scores(best_random_full, X_train_copy, y_train_copy)

The precision score is : 0.9596070916221314
The recall score is: 0.9595666666666667

```

The f1 score is: 0.959541389797958

The precision score is : 0.9637253089368318

The recall score is: 0.96371

The f1 score is: 0.9636919641273892

```
best_random_full.fit(X_train_copy, y_train_copy)
```

```
judge_scores(best_random_full, X_train_copy, y_train_copy)
```

The precision score is : 1.0

The recall score is: 1.0

The f1 score is: 1.0

The precision score is : 0.9637253089368318

The recall score is: 0.96371

The f1 score is: 0.9636919641273892

```
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
X_train_copy_scaled =
scaler.fit_transform(X_train_copy.astype(np.float64))
random_forest_best_scaled_cross_val_scores =
cross_val_score(best_random_full, X_train_copy_scaled, y_train_copy,
scoring="accuracy", n_jobs=-1)
print(random_forest_best_scaled_cross_val_scores)
```


_RemoteTraceback Traceback (most recent call last)

_RemoteTraceback:
"""

Traceback (most recent call last):

File "d:\Code Playground\Python\AI_ML\classificationDigits\.venv\Lib\site-packages\joblib\externals\loky\process_executor.py", line 463, in _process_worker

```
    r = call_item()
        ^^^^^^^^^^^
```

File "d:\Code Playground\Python\AI_ML\classificationDigits\.venv\Lib\site-packages\joblib\externals\loky\process_executor.py", line 291, in __call__

```
    return self.fn(*self.args, **self.kwargs)
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
```

File "d:\Code Playground\Python\AI_ML\classificationDigits\.venv\Lib\site-packages\joblib\parallel.py", line 598, in __call__

```
    return [func(*args, **kwargs)
            ^^^^^^^^^^^^^^^^^^^
```

File "d:\Code Playground\Python\AI_ML\classificationDigits\.venv\Lib\site-packages\sklearn\utils\parallel.py", line 139, in __call__

```
    return self.function(*args, **kwargs)
```



```

214         )
215     ):
--> 216         return func(*args, **kwargs)
217 except InvalidParameterError as e:
218     # When the function is just a wrapper around an estimator,
we allow
219     # the function to delegate validation to the estimator,
but we replace
220     # the name of the estimator by the name of the function in
the error
221     # message to avoid confusion.
222     msg = re.sub(
223         r"parameter of \w+ must be",
224         f"parameter of {func.__qualname__} must be",
225         str(e),
226     )

```

File d:\Code Playground\Python\AI_ML\classificationDigits\.venv\Lib\site-packages\sklearn\model_selection_validation.py:684, in cross_val_score(estimator, X, y, groups, scoring, cv, n_jobs, verbose, params, pre_dispatch, error_score)

```

681 # To ensure multimetric format is not supported
682 scorer = check_scoring(estimator, scoring=scoring)
--> 684 cv_results = cross_validate(
685     estimator=estimator,
686     X=X,
687     y=y,
688     groups=groups,
689     scoring={"score": scorer},
690     cv=cv,
691     n_jobs=n_jobs,
692     verbose=verbose,
693     params=params,
694     pre_dispatch=pre_dispatch,
695     error_score=error_score,
696 )
697 return cv_results["test_score"]

```

File d:\Code Playground\Python\AI_ML\classificationDigits\.venv\Lib\site-packages\sklearn\utils_param_validation.py:216, in validate_params.<locals>.decorator.<locals>.wrapper(*args, **kwargs)

```

210 try:
211     with config_context(
212         skip_parameter_validation=(
213             prefer_skip_nested_validation or
global_skip_validation
214         )
215     ):
--> 216         return func(*args, **kwargs)

```

```

217 except InvalidParameterError as e:
218     # When the function is just a wrapper around an estimator,
we allow
219     # the function to delegate validation to the estimator,
but we replace
220     # the name of the estimator by the name of the function in
the error
221     # message to avoid confusion.
222     msg = re.sub(
223         r"parameter of \w+ must be",
224         f"parameter of {func.__qualname__} must be",
225         str(e),
226     )

```

File d:\Code Playground\Python\AI_ML\classificationDigits\.venv\Lib\site-packages\sklearn\model_selection_validation.py:411, in cross_validate(estimator, X, y, groups, scoring, cv, n_jobs, verbose, params, pre_dispatch, return_train_score, return_estimator, return_indices, error_score)

```

408 # We clone the estimator to make sure that all the folds are
409 # independent, and that it is pickle-able.
410 parallel = Parallel(n_jobs=n_jobs, verbose=verbose,
pre_dispatch=pre_dispatch)
--> 411 results = parallel(
412     delayed(_fit_and_score)(
413         clone(estimator),
414         X,
415         y,
416         scorer=scorers,
417         train=train,
418         test=test,
419         verbose=verbose,
420         parameters=None,
421         fit_params=routed_params.estimator.fit,
422         score_params=routed_params.scorer.score,
423         return_train_score=return_train_score,
424         return_times=True,
425         return_estimator=return_estimator,
426         error_score=error_score,
427     )
428     for train, test in indices
429 )
431 _warn_or_raise_about_fit_failures(results, error_score)
433 # For callable scoring, the return type is only know after
calling. If the
434 # return type is a dictionary, the error scores can now be
inserted with
435 # the correct key.

```



```
File d:\Code Playground\Python\AI_ML\classificationDigits\.venv\Lib\
site-packages\sklearn\utils\parallel.py:77, in Parallel.__call__(self,
iterable)
```

```
    72 config = get_config()
    73 iterable_with_config = (
    74     (_with_config(delayed_func, config), args, kwargs)
    75     for delayed_func, args, kwargs in iterable
    76 )
--> 77 return super().__call__(iterable_with_config)
```

```
File d:\Code Playground\Python\AI_ML\classificationDigits\.venv\Lib\
site-packages\joblib\parallel.py:2007, in Parallel.__call__(self,
iterable)
```

```
    2001 # The first item from the output is blank, but it makes the
interpreter
```

```
    2002 # progress until it enters the Try/Except block of the
generator and
```

```
    2003 # reaches the first `yield` statement. This starts the
asynchronous
```

```
    2004 # dispatch of the tasks to the workers.
```

```
    2005 next(output)
```

```
-> 2007 return output if self.return_generator else list(output)
```

```
File d:\Code Playground\Python\AI_ML\classificationDigits\.venv\Lib\
site-packages\joblib\parallel.py:1650, in Parallel._get_outputs(self,
iterator, pre_dispatch)
```

```
    1647     yield
    1649     with self._backend.retrieval_context():
-> 1650         yield from self._retrieve()
    1652 except GeneratorExit:
    1653     # The generator has been garbage collected before being
fully
    1654     # consumed. This aborts the remaining tasks if possible
and warn
    1655     # the user if necessary.
    1656     self._exception = True
```

```
File d:\Code Playground\Python\AI_ML\classificationDigits\.venv\Lib\
site-packages\joblib\parallel.py:1754, in Parallel._retrieve(self)
```

```
    1747 while self._wait_retrieval():
    1748
    1749     # If the callback thread of a worker has signaled that its
task
    1750     # triggered an exception, or if the retrieval loop has
raised an
    1751     # exception (e.g. `GeneratorExit`), exit the loop and
surface the
    1752     # worker traceback.
    1753     if self._aborting:
-> 1754         self._raise_error_fast()
```

```
1755         break
1757     # If the next job is not ready for retrieval yet, we just
wait for
1758     # async callbacks to progress.
```

```
File d:\Code Playground\Python\AI_ML\classificationDigits\.venv\Lib\
site-packages\joblib\parallel.py:1789, in
Parallel._raise_error_fast(self)
```

```
1785 # If this error job exists, immediately raise the error by
1786 # calling get_result. This job might not exist if abort has
been
1787 # called directly or if the generator is gc'ed.
1788 if error_job is not None:
-> 1789     error_job.get_result(self.timeout)
```

```
File d:\Code Playground\Python\AI_ML\classificationDigits\.venv\Lib\
site-packages\joblib\parallel.py:745, in
BatchCompletionCallback.get_result(self, timeout)
```

```
739 backend = self.parallel._backend
741 if backend.supports_retrieve_callback:
742     # We assume that the result has already been retrieved by
the
743     # callback thread, and is stored internally. It's just
waiting to
744     # be returned.
--> 745     return self._return_or_raise()
747 # For other backends, the main thread needs to run the
retrieval step.
748 try:
```

```
File d:\Code Playground\Python\AI_ML\classificationDigits\.venv\Lib\
site-packages\joblib\parallel.py:763, in
BatchCompletionCallback._return_or_raise(self)
```

```
761 try:
762     if self.status == TASK_ERROR:
--> 763         raise self._result
764     return self._result
765 finally:
```

```
MemoryError: Unable to allocate 1.40 GiB for an array with shape
(240000, 784) and data type float64
```